

TASK3 策略首秀：用均线交叉反应市场趋势变化

在量化交易领域，交易策略是实现盈利的核心。本任务旨在深入掌握经典且广泛应用的量化交易策略——双均线策略，并通过实践操作，学会如何利用均线交叉信号（金叉与死叉）来捕捉市场趋势变化，而后再进行交易决策。此外，最大回撤 (MDD)，夏普比率 (Sharpe Ratio)、累计回报 (Cumulative Return) 等关键指标能进一步帮助全面评估交易策略的性能，从而为后续更复杂策略的开发和优化积累经验。

任务 1. 了解双均线策略，对其中的金叉，死叉等概念做出解释

双均线策略 (Dual Moving Average Strategy) 属于趋势跟踪型量化交易策略，核心思想是利用两条不同周期的移动平均线判断市场趋势，并根据两条均线之间的位置关系产生买卖信号。策略通常选择一条周期较短的均线作为短期均线 (Short-term Moving Average)，用于反映市场近期价格变化；另一条周期较长的均线作为长期均线 (Long-term Moving Average)，用于反映市场整体趋势。例如，常见的组合包括 5 日均线与 20 日均线、10 日均线与 30 日均线以及 20 日均线与 60 日均线等。当短期均线突破长期均线时，说明近期价格上涨速度快于长期平均水平，市场可能进入上涨趋势；当短期均线跌破长期均线时，则意味着市场可能进入下跌趋势。因此，双均线策略通过比较短期与长期价格趋势之间的相对变化，尽量在趋势形成初期买入，在趋势结束时卖出，从而获取趋势行情带来的收益。

金叉 (Golden Cross) 是双均线策略中的重要买入信号，是指短期均线由下向上突破长期均线形成交叉。当市场经历一段下跌或横盘后，若近期价格持续上涨，短期均线会逐渐向上移动，并最终突破长期均线，此时形成金叉。金叉意味着短期市场表现已经强于长期平均水平，说明市场可能进入新的上涨趋势，投资者通常将其视为买入或建仓信号。在实际交易中，为了减少价格短期波动带来的虚假信号，许多策略还会结合成交量、趋势强度或其他技术指标共同确认金叉是否有效。

死叉 (Death Cross) 是双均线策略中的重要卖出信号，是指短期均线由上向下跌破长期均线形成交叉。当市场上涨趋势减弱或价格持续下跌时，短期均线会逐渐向下移动，并跌破长期均线，此时形成死叉。死叉表明近期价格已经弱于长期平均水平，市场可能进入下跌趋势，因此投资者通常将其作为卖出、减仓或止损的重要依据。与金叉类似，为避免震荡行情中频繁产生错误交易信号，实际量化策略通常也会设置一定的确认条件，例如要求交叉持续若干个交易日或结合其他指标共同判断。

持仓 (Position) 是指投资者当前是否持有某只资产以及持有的数量。在双均线策略中, 交易系统通常只有两种基本状态: 空仓 (Flat Position) 和持仓 (Long Position)。当策略检测到金叉信号时, 会执行买入操作, 使账户由空仓转为持仓; 当检测到死叉信号时, 则执行卖出操作, 使账户重新回到空仓状态。整个交易过程实际上就是在这两种状态之间不断切换, 从而实现趋势跟踪。

买入信号 (Buy Signal) 是指策略认为未来价格具有上涨概率, 因此建议建立仓位的交易信号; **卖出信号 (Sell Signal)** 则表示策略认为未来价格可能下跌, 因此建议卖出持仓或退出市场。在双均线策略中, 金叉通常对应买入信号, 死叉通常对应卖出信号。量化交易系统会根据这些预先定义好的规则自动执行交易, 从而避免人为情绪对投资决策产生影响, 提高交易的一致性和纪律性。

任务 2. 了解量化策略效果的评估指标, 对其中最大回撤 (MDD)、夏普比率 (Sharpe Ratio)、累计回报 (Cumulative Return) 等概念做出解释

在量化交易中, 仅观察策略最终获得多少收益并不足以全面评价策略的优劣, 还需要综合考虑策略承担的风险、收益稳定性以及资金回撤情况。因此, 量化投资通常采用多种评价指标对策略进行综合分析。其中, 累计回报 (Cumulative Return) 用于衡量策略整体获得的收益水平, 是评价盈利能力最直观的指标; 最大回撤 (Maximum Drawdown, MDD) 用于衡量策略在运行过程中可能遭受的最大资金损失, 是评价风险控制能力的重要指标; 夏普比率 (Sharpe Ratio) 则综合考虑收益与风险之间的关系, 用于评价策略在承担单位风险时所获得的超额收益, 是衡量风险调整后收益能力的重要指标。这三项指标分别从收益、风险以及收益风险比三个不同角度评价量化策略, 因此也是学术研究和实际投资中应用最广泛的策略评价指标。

1. 最大回撤 MDD

最大回撤 (Maximum Drawdown, MDD) 指策略净值在整个运行期间, 从某一历史最高点下降到之后最低点所经历的最大跌幅, 其计算方式通常表示为历史峰值与随后最低净值之间的最大下降比例, 即

$$MDD = \max_t \left(\frac{Peak_t - Trough_t}{Peak_t} \right) \quad (1)$$

其中, $\text{Peak}_t = \max_{0 \leq i \leq t} V_i$ 表示截至时刻 t 之前策略净值达到的历史最高值, $\text{Trough}_t = \min_{j \geq t} V_j$ 表示该峰值之后出现的最低净值。最大回撤越小, 说明策略在市场波动过程中资金损失越有限, 风险控制能力越强; 反之, 若最大回撤较大, 则意味着投资者可能需要承受较大的阶段性亏损。对于大多数投资者而言, 最大回撤往往比收益更加重要, 因为即使策略长期收益较高, 若期间出现过大的资金回撤, 也可能导致投资者无法继续持有策略。因此, 在实际量化投资中, 最大回撤通常被视为衡量策略稳定性和风险控制能力的重要指标。

2. 夏普比率 Sharpe Ratio

夏普比率 (Sharpe Ratio) 是由诺贝尔经济学奖获得者威廉·夏普提出的一种风险调整收益指标, 用于衡量策略每承担一单位风险所获得的超额收益, 其定义为

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p} \quad (2)$$

其中, R_p 表示策略平均收益率, R_f 表示无风险收益率 (例如国债收益率), σ_p 表示策略收益率的标准差, 用于衡量收益波动程度。夏普比率越高, 说明策略在承担相同风险的情况下能够获得更高的超额收益, 资金利用效率越高; 若夏普比率较低, 则说明策略收益不足以补偿所承担的风险。在量化投资中, 夏普比率能够综合考虑收益与波动性, 因此比单纯观察收益率更具有参考价值。例如, 两种策略都获得 20% 的年化收益, 但其中一种收益波动明显较小, 则其夏普比率通常更高, 也意味着该策略具有更好的稳定性和投资价值。

3. 累计回报 Cumulative Return

累计回报 (Cumulative Return) 用于衡量策略在整个投资期间获得的总体收益水平, 是评价策略盈利能力最直接的指标。假设策略初始资金为 V_0 , 期末资金为 V_T , 则累计回报可表示为

$$\text{Cumulative Return} = \frac{V_T - V_0}{V_0} \quad (3)$$

例如, 若投资者初始资金为 100 万元, 最终资金增长至 150 万元, 则累计回报为 50%。累计回报能够直观反映策略最终创造了多少收益, 因此常用于不同策略之间的收益比较。然而, 仅依赖累计回报评价策略存在一定局限性, 因为两种策略即使获得相同的累计收益, 其承担的风险可能截然不同。例如, 一种策略收益增长平稳, 而另一种策略在获得相同收益的过程

中经历了较大的资金波动和回撤。因此，在实际评价量化策略时，累计回报通常需要与最大回撤、夏普比率等风险指标结合分析，才能更加全面地评价策略的实际表现。

任务 3. python 编程实现以下内容:

- 1) 设定短均线和长均线的周期，如 5 和 15，计算均线数据
- 2) 计算买入卖出的交易信号
- 3) 根据计算结果绘制对应可视化图形，包括股价，长短均线，交易信号，标记买入信号，卖出信号
- 4) 对策略进行模拟交易和回测，计算以上量化指标

```
def run_backtest(  
    df: pd.DataFrame,  
    short_period: int = DEFAULT_SHORT_SMA,  
    long_period: int = DEFAULT_LONG_SMA,  
    initial_capital: float = DEFAULT_INITIAL_CAPITAL,  
    commission_rate: float = 0.0,  
    start_date: str | None = None,  
    end_date: str | None = None,  
    ) -> dict:
```

"""

运行双均线策略回测。

参数

df : pd.DataFrame 原始日线数据，需包含 trade_date 和 close 列

short_period : int 短周期 SMA

long_period : int 长周期 SMA

initial_capital : float 初始资金（元）

commission_rate : float 手续费率（例如 0.0003 表示万分之三）。

 买入时：每股成本 = 价格 × (1 + rate)

 卖出时：每股收入 = 价格 × (1 - rate)

 设为 0 则不计手续费。

start_date : str | None 回测起始日期（YYYYMMDD 或 YYYY-MM-DD），None 表示数据最早日期

end_date : str | None 回测结束日期（YYYYMMDD 或 YYYY-MM-DD），None 表示数据最晚日期

返回

```
data : dict
    dates, close, short_ma, long_ma,
    buy_signals [{date, price}], sell_signals [{date, price}],
    portfolio_value, benchmark_value, drawdown
metrics : dict
    total_return, annual_return, sharpe_ratio,
    max_drawdown, max_drawdown_date,
    win_rate, trade_count, profit_loss_ratio
"""

# 数据准备
df = df.copy()
if "trade_date" in df.columns:
    if not
pd.api.types.is_datetime64_any_dtype(df["trade_date"]):
    df["trade_date"] =
pd.to_datetime(df["trade_date"].astype(str), format="%Y%m%d")
df = df.sort_values("trade_date").reset_index(drop=True)

close_series = df.set_index("trade_date")["close"]

# 日期过滤
if start_date is not None:
    start_dt = pd.to_datetime(start_date)
    close_series = close_series[close_series.index >= start_dt]
if end_date is not None:
    end_dt = pd.to_datetime(end_date)
    close_series = close_series[close_series.index <= end_dt]

n = len(close_series)
if n == 0:
    raise ValueError("指定的日期范围内无交易数据")

min_required = max(short_period, long_period) + 1
if n < min_required:
    raise ValueError(
        f"数据不足：回测区间仅 {n} 个交易日，"
```

```

        f"至少需要 {min_required} 天 (max(SMA 周期) + 1) "
    )

# 计算信号
sig_df = compute_signals(close_series, short_period,
long_period)
short_ma = sig_df["short_ma"]
long_ma = sig_df["long_ma"]
signals = sig_df["signal"]

# 模拟交易
buy_cost_factor = 1.0 + commission_rate # 买入成本系数
sell_proceed_factor = 1.0 - commission_rate # 卖出收入系数

cash = float(initial_capital)
shares = 0.0
portfolio_values = np.zeros(n)
trades = [] # 记录每笔完整交易

current_buy_date = None
current_buy_price = None
current_buy_cost = None # 本次买入总成本

for i in range(n):
    price = float(close_series.iloc[i])
    sig = signals.iloc[i]

    # 金叉买入
    if sig == 1 and shares == 0.0 and cash > 0.0:
        cost_per_share = price * buy_cost_factor
        if cost_per_share > 0:
            shares = cash / cost_per_share
            cash = 0.0
            current_buy_date = close_series.index[i]
            current_buy_price = price
            current_buy_cost = shares * cost_per_share

    # 死叉卖出

```

```

elif sig == -1 and shares > 0.0:
    proceeds = shares * price * sell_proceed_factor
    cash = proceeds
    if current_buy_date is not None and current_buy_cost is not
None and current_buy_cost > 0:
        trade_return = (proceeds - current_buy_cost) /
current_buy_cost
        trades.append({
            "buy_date": str(current_buy_date.date()),
            "buy_price": round(current_buy_price, 2),
            "sell_date": str(close_series.index[i].date()),
            "sell_price": round(price, 2),
            "return": round(float(trade_return), 6),
            "win": trade_return > 0,
        })
    shares = 0.0
    current_buy_date = None
    current_buy_price = None
    current_buy_cost = None

# 当日组合净值
portfolio_values[i] = cash + shares * price

# 回测结束时强制平仓
if shares > 0.0:
    last_price = float(close_series.iloc[-1])
    last_date = close_series.index[-1]
    proceeds = shares * last_price * sell_proceed_factor
    cash = proceeds
    if current_buy_date is not None and current_buy_cost is not None
and current_buy_cost > 0:
        trade_return = (proceeds - current_buy_cost) /
current_buy_cost
        trades.append({
            "buy_date": str(current_buy_date.date()),
            "buy_price": round(current_buy_price, 2) if
current_buy_price else 0,
            "sell_date": str(last_date.date()),

```

```

        "sell_price": round(last_price, 2),
        "return": round(float(trade_return), 6),
        "win": trade_return > 0,
    })
    shares = 0.0
    portfolio_values[-1] = cash

    pv_series = pd.Series(portfolio_values,
index=close_series.index)

# 基准（买入持有）
benchmark_shares = initial_capital / float(close_series.iloc[0])
benchmark_values = close_series.values * benchmark_shares

# 回撤
running_max = np.maximum.accumulate(portfolio_values)
drawdown_pct = (portfolio_values / running_max - 1.0) * 100.0

# 每日收益率
daily_returns = pv_series.pct_change().dropna()

# 指标计算
total_days = (close_series.index[-1] -
close_series.index[0]).days
n_years = total_days / 365.25 if total_days > 0 else 0.0

final_value = float(portfolio_values[-1])
total_return = (final_value / initial_capital - 1.0) * 100.0

# 年化收益率
if n_years > 0 and final_value > 0:
    annual_return = ((final_value / initial_capital) ** (1.0 /
n_years) - 1.0) * 100.0
else: annual_return = None

# 夏普比率（无风险利率设为 0）
if len(daily_returns) > 1:
    std_ret = float(daily_returns.std())

```



```

        if std_ret > 1e-12: sharpe = float(daily_returns.mean() /
std_ret * np.sqrt(252))
        else: sharpe = 0.0
    else: sharpe = None

# 最大回撤
max_dd = float(np.min(drawdown_pct))
max_dd_idx = int(np.argmin(drawdown_pct))
max_dd_date = str(close_series.index[max_dd_idx].date())

# 胜率 & 盈亏比
if trades:
    wins = [t for t in trades if t["win"]]
    losses = [t for t in trades if not t["win"]]
    win_rate = len(wins) / len(trades) * 100.0
    avg_win = float(np.mean([t["return"] for t in wins])) if wins
else 0.0
    avg_loss = abs(float(np.mean([t["return"] for t in losses])))
if losses else 0.0
    pl_ratio = avg_win / avg_loss if avg_loss > 1e-12 else None
else:
    win_rate = 0.0
    pl_ratio = None

# 基准最终收益
benchmark_final = float(benchmark_values[-1])
benchmark_return = (benchmark_final / initial_capital - 1.0) *
100.0

# 收集买卖信号点
buy_signals = []
sell_signals = []
for i in range(n):
    sig = signals.iloc[i]
    if sig == 1:
        buy_signals.append({
            "date": str(close_series.index[i].date()),
            "price": round(float(close_series.iloc[i]), 2),

```

```

    })
elif sig == -1:
    sell_signals.append({
        "date": str(close_series.index[i].date()),
        "price": round(float(close_series.iloc[i]), 2),
    })

```

回测结果（看板）：



图 1 双均线策略回测看板

任务 4. 尝试不同股票，均线周期，观察收益情况变化，总结双均线策略适用场景和心得

1) 同一标的，不同均线周期

针对寒武纪标的，分别使用均线周期(5,20)和(20,60)两种组合回测 22-23 及 24-25 数据。

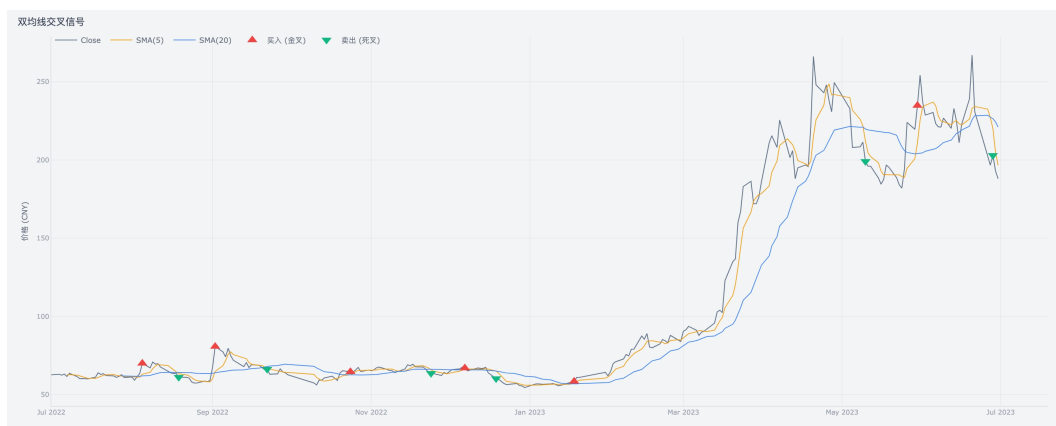




图 2 220701 至 230701 寒武纪均线周期(5,20)与(20,60)对比



图 3 240701 至 250701 寒武纪均线周期(5,20)与(20,60)对比

总体来看，较短周期的均线组合能够更快地响应价格变化，当股价出现上涨趋势时，短周期均线会更早向上突破长周期均线，从而提前发出买入信号；当趋势结束时，也能够更早地产生卖出信号。本次回测中，短周期均线组合的累计收益率整体高于长周期均线组合，也验证了这一特点。

然而，均线周期的缩短也带来了更多的无效交易，在震荡行情中容易出现频繁的“金叉—死叉”切换，从而提高交易成本并降低策略稳定性。相比之下，较长周期均线能够对价格

进行更充分的平滑处理，过滤掉部分短期波动，虽然买卖信号存在一定滞后，但能够减少频繁交易，使策略更加稳健。从图上也能够看到很多笔交易其实并没有带来收益。

总结来说，均线周期的选择本质上是在信号灵敏度与稳定性之间进行权衡。对于趋势较为明显、波动连续的市场环境，可以适当采用较短的均线周期以提高收益；而对于震荡较多或交易成本较高的市场，则应适当延长均线周期，以降低无效交易带来的影响。

2) 不同标的，相同策略

针对寒武纪和贵州茅台，使用均线周期(5,20)策略回测 21-26 数据。图 4 上半部分为寒武纪数据，下半部分为贵州茅台数据。



图 4 20210701 至 20260701 不同类型标的收益对比

表 1 20210701 至 20260701 不同标的主要指标对比

标的	年化收益率 (%)	夏普比率	最大回撤 (%)	胜率 (%)
寒武纪	57.38	1.09	-56.70	36.40
贵州茅台	-6.41	-0.35	-45.33	25.00

在保持双均线参数完全一致的情况下，不同股票标的的策略表现存在明显差异。本次实验分别对寒武纪和贵州茅台进行了回测，结果表明，双均线策略在寒武纪上的收益明显优于贵州茅台。结合价格走势可以发现，寒武纪在回测期间整体呈现出较为明显的单边上涨趋势，股价上涨阶段持续时间较长，趋势特征清晰。当短期均线向上突破长期均线时，策略能够较早建立仓位，并在上涨趋势持续期间一直持有；当趋势结束、均线形成死叉时，再及时退出市场。从回测图中可以直观地观察到，策略基本能够在每一轮主要上涨行情的初期完成买入，在上涨结束后完成卖出，因此获得了较好的累计收益。

相比之下，贵州茅台在回测区间内的价格走势相对更加平稳，阶段性震荡较多，缺乏持续且显著的单边趋势。在这种市场环境下，均线交叉更多反映的是价格的短期波动，而非真正的趋势转换，容易产生较多滞后的交易信号，导致策略难以获得持续收益。这印证了双均线策略本质上属于一种趋势跟踪策略，其盈利能力高度依赖于标的是否具有持续的趋势行情。当股票处于明显的上涨或下跌趋势时，策略能够充分发挥“让利润奔跑”的优势；而在横盘震荡市场中，则容易受到频繁交叉信号的影响，出现收益下降甚至连续亏损。因此，在实际应用中，应优先选择趋势性较强、方向明确的股票或市场环境来使用双均线策略，并结合趋势强度指标或波动率指标对震荡行情进行过滤，以进一步提升策略的稳定性和收益表现。